Universidad Nacional Autónoma de México
Facultad de Ingeniería

Criptografía

Practical Session A – Creating Asymmetric Keys and Digital Certificates

Silverio Martínez Andrés

PROFESORA: DRA. Rocío Alejandra Aldeco Pérez

Semestre 2026-1

Grupo: 1

Fecha de entrega: 25 de noviembre de 2025

Practical Session A – Creating Asymmetric Keys and Digital Certificates

Using the OpenSSL Command Line Utilities, do the following tasks:

1. An RSA Private Key (2048 bits)
2. An RSA Public Key (2048 bits)
3. With the previous generated private key créate a self-signed certificate. Use the pkcs12 format.
4. With the same key, créate a certificate that is signed by an authority. Use the pkcs12 format. Use the free services Let's Encrypt or Getacert.
5. Verify this last certificate

As evidence of these processes create a report that explains which commands you used to achieve each of the tasks, add the corresponding screenshots and explain the process you followed. Add your conclusions and upload it to Canvas.

Desarrollo

Antes de empezar con la práctica, es necesario instalar OpenSSL en mi computadora, para ello (y como no tenía Git en mi computadora), descargué Git, ya que OpenSSL viene incluido con Git de manera predeterminada. Para ello, lo descargué utilizando el gestor de paquetes de Microsoft con PowerShell, con el siguiente comando:

```
PS C:\Users\andik> winget install Git.Git
```

Una vez descargado, abrí "Git Bash" y procedí a comprobar que OpenSSL estaba instalado de la siguiente manera, con el comando "openssl version":

```
MINGW64:/c/Users/andik

andik@KylosPC MINGW64 ~
$ openssl version
OpenSSL 3.5.4 30 Sep 2025 (Library: OpenSSL 3.5.4 30 Sep 2025)
```

Mostrando que OpenSSL ya estaba descargado en su versión 3.5.4.

Como primer punto de la práctica se solicita que, utilizando la línea de comandos de OpenSSL:

1. Crear una clave privada RSA (2048 bits)

Donde, revisando un poco la documentación con los comandos que se pueden utilizar en OpenSSL, disponible en https://docs.openssl.org/master/man1/#options

encontré el siguiente, el cuales sería de ayuda para poder completar este primer punto de mi práctica, siendo:

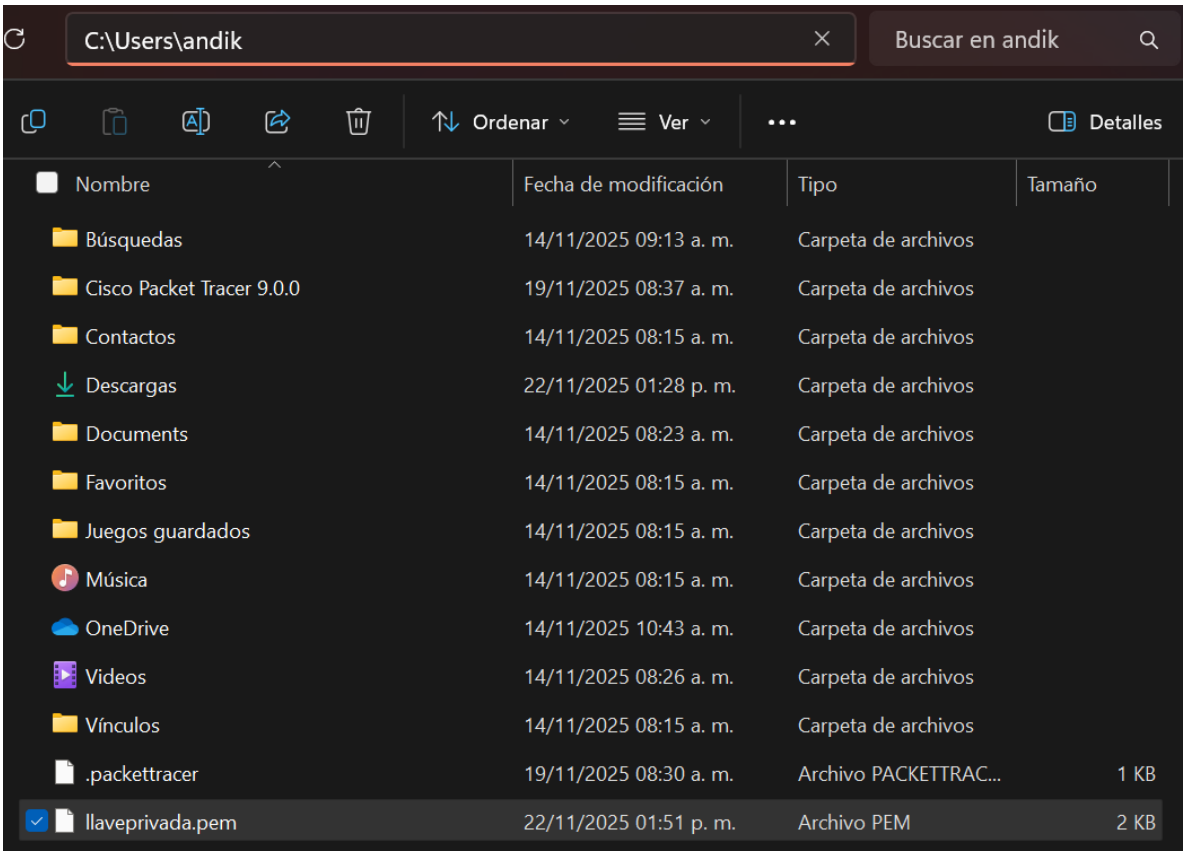
- genrsa -> Siendo este el subcomando que ayuda a generar una clave privada en RSA

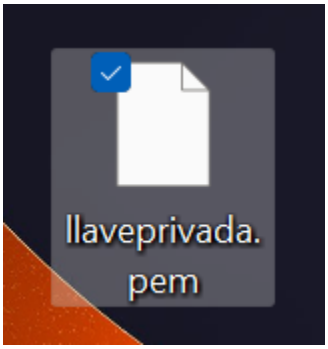
Ahora bien, como quiero que mi llave privada tenga un nombre en específico, a lado de “genrsa”, añadí un “-out llaveprivada.pem”, para que, al correr el comando, se genere mi llave privada con nombre “llaveprivada”. Por último, se adjuntará el número “2048”, con el fin de indicar que mi llave privada tendrá un tamaño de 2048 bits. Entonces, una vez sabiendo que el comando a utilizar es “openssl genrsa -out llaveprivada.pem 2048”, procedo a probarlo en Git Bash.

```
andik@KylosPC MINGW64 ~
$ openssl genrsa -out llaveprivada.pem 2048
```

Esto ya generó la llave privada RSA, con tamaño de 2048 bits, y con nombre “llaveprivada”, para encontrarla, es necesario saber en que directorio estoy trabajando desde Git Bash, para ello, con el comando “pwd”, muestro el directorio donde se creó mi llave pública, y la muevo al escritorio (para mayor practicidad).

```
andik@KylosPC MINGW64 ~
$ pwd
/c/Users/andik
```





Confirmando que se ha generado la llave privada RSA.

Ahora bien, para el segundo punto de la práctica se solicitó que:

2. Generar una clave pública RSA (2048 bits)

Donde igual, revisando un poco la documentación de los comandos que se pueden utilizar en OpenSSL, encontré el siguiente:

- rsa -> subcomando que se utiliza para procesar, verificar y convertir claves privadas y públicas RSA.

Y que, con ayuda de los subcomandos:

- “-in llaveprivada.pem”: Indica el archivo de entrada de donde quiero extraer la clave pública (que es llaveprivada.pem)
- “-pubout”: El cual le indica al subcomando “rsa” que solo debe generar y mostrar la clave pública
- “-out llavepublica.pem”: Para que el archivo de salida lleve dicho nombre.

Antes de correr el comando, desde Git Bash, me moveré de directorio hasta el escritorio, para no repetir los pasos anteriores de estar buscando en donde se guardo el archivo.

```
andik@KylosPC MINGW64 ~
$ cd desktop

andik@KylosPC MINGW64 ~/desktop
$
```

Ya con esto, el archivo automáticamente se guardará en el escritorio.

Ahora sí, procedo a utilizar el comando “openssl rsa -in llaveprivada.pem -pubout -out llavepublica.pem”, teniendo como resultado lo siguiente:

```
andik@KylosPC MINGW64 ~/desktop
$ openssl rsa -in llaveprivada.pem -pubout -out llavepublica.pem
writing RSA key
```

Creando así, mi clave pública a partir de mi clave privada.

Ahora bien, para el tercer paso de la práctica se solicita el:

3. Crear un certificado autofirmado a partir de la clave privada generada anteriormente. Utiliza el formato pkcs12

Investigando un poco, vi que, para crear un certificado autofirmado a partir de una clave privada, utilizando el formato que se nos solicita, es necesario, primero, crear el certificado x.509 (crt), y luego convertirlo a pkcs12, ya que pkcs12 es un contenedor seguro que almacena tanto el certificado como la clave privada juntos,

por eso es neceario primero generar el certificado, y ya luego empaquetarlos con pkcs12.

Entonces, primero se genera el certificado x.509, esto lográndolo con el subcomando:

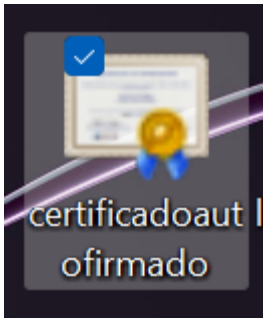
- req -> Subcomando utilizado para gestionar solicitudes de firma de certificado (csr) y, con opciones especiales para crear certificados autofirmados.

Y que, junto con los subcomandos:

- “-new -x509”: Indican que se va a crear una nueva solicitud de certificado, así como que el certificado a generar será un x.509 (el cual está autofirmado)
- “-key llaveprivada.pem”: Indica el archivo de la clave privada que se usará para firmar el certificado.
- “-out certificadoautofirmado.crt”: Define el archivo de salida donde se guardará el certificado generado.
- “-days 365”: Indica la validez del certificado en días a partir de la fecha de creación.

Una vez sabiendo que subcomandos se generaran para poder lograr la creación del certificado auto firmado, procedí a probarlo.

```
andik@KylosPC MINGW64 ~/desktop
$ openssl req -new -x509 -key llaveprivada.pem -out certificadoautofirmado.crt -
days 365
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:MX
State or Province Name (full name) [Some-State]:CDMX
Locality Name (eg, city) []:Iztapalapa
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:Andres
Email Address []:kylosarrollo1234@gmail.com
```



Donde, una vez puesto el comando, y haber dado “enter”, pedirá ingresar varia información, con el propósito de identificar al propietario del certificado.

Ya teniendo el certificado auto firmado, solo queda empaquetar la llave privada con el certificado, y en formato “pkcs12”, esto haciendo uso del siguiente subcomando:

- “pkcs12” -> Subcomando que gestiona archivos de formato pkcs12. Almacena objetos criptográficos en un solo archivo.

Y que con ayuda de los siguientes subcomandos, se empaquetarán el certificado auto firmado y la clave privada:

- “-export”: Indica que se debe crear un archivo pkcs12 a partir de los componentes de entrada.
- “-out certificadoautofirmado.p12”: Define el archivo de salida, siendo de formato contenedor binario.
- “-inkey llaveprivada.pem”: Especifica el archivo de la clave privada que se incluirá en el contenedor.
- “-in certificadoautofirmado.crt”: Especifica el archivo de certificado digital que también se incluirá en el contenedor.

Una vez teniendo el comando completo, lo corrí, y salió de la siguiente manera:

```
andik@KylosPC MINGW64 ~/Desktop
$ openssl pkcs12 -export -out certificadoautofirmado.p12 -inkey llaveprivada.pem
 -in certificadoautofirmado.crt
Enter Export Password:

Verifying - Enter Export Password:
```

Donde al dar enter, te pide que ingreses una contraseña, yo le puse “kylos”.

Ahora bien, para el cuarto punto de la práctica, se pide lo siguiente:

4. Con la misma llave, crea un certificado firmado por una autoridad. Usa el formato pkcs12. Utiliza los servicios gratuitos Let’s Encrypt o Getacert.

Para ello, se harán 3 pasos:

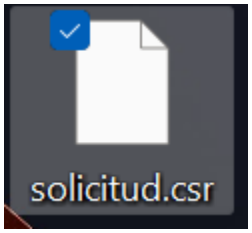
1. Crear una solicitud de firma de certificado (csr)

Para ello, utilicé el comando “openssl req -new -key llaveprivada.pem -out solicitud.csr”, para crear el archivo “solicitud.csr” a partir de la clave privada creada en un inicio.

Quedando de la siguiente manera:

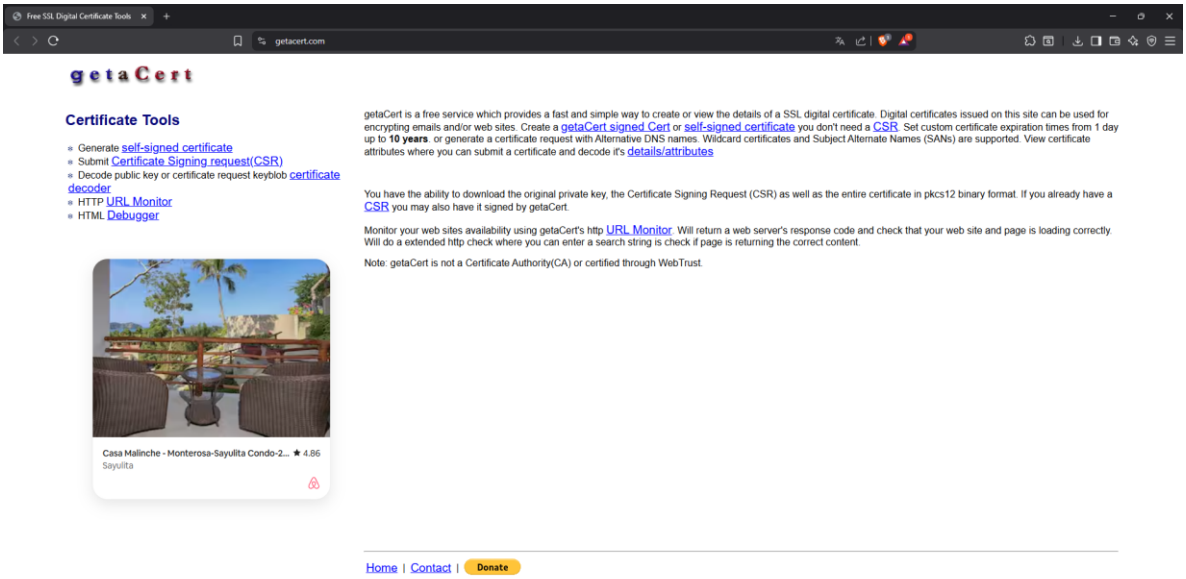
```
andik@KylosPC MINGW64 ~/Desktop
$ openssl req -new -key llaveprivada.pem -out solicitud.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:MX
State or Province Name (full name) [Some-State]:CDMX
Locality Name (eg, city) []:Iztapalapa
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:Andres
Email Address []:kylosarrollo1234@gmail.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```

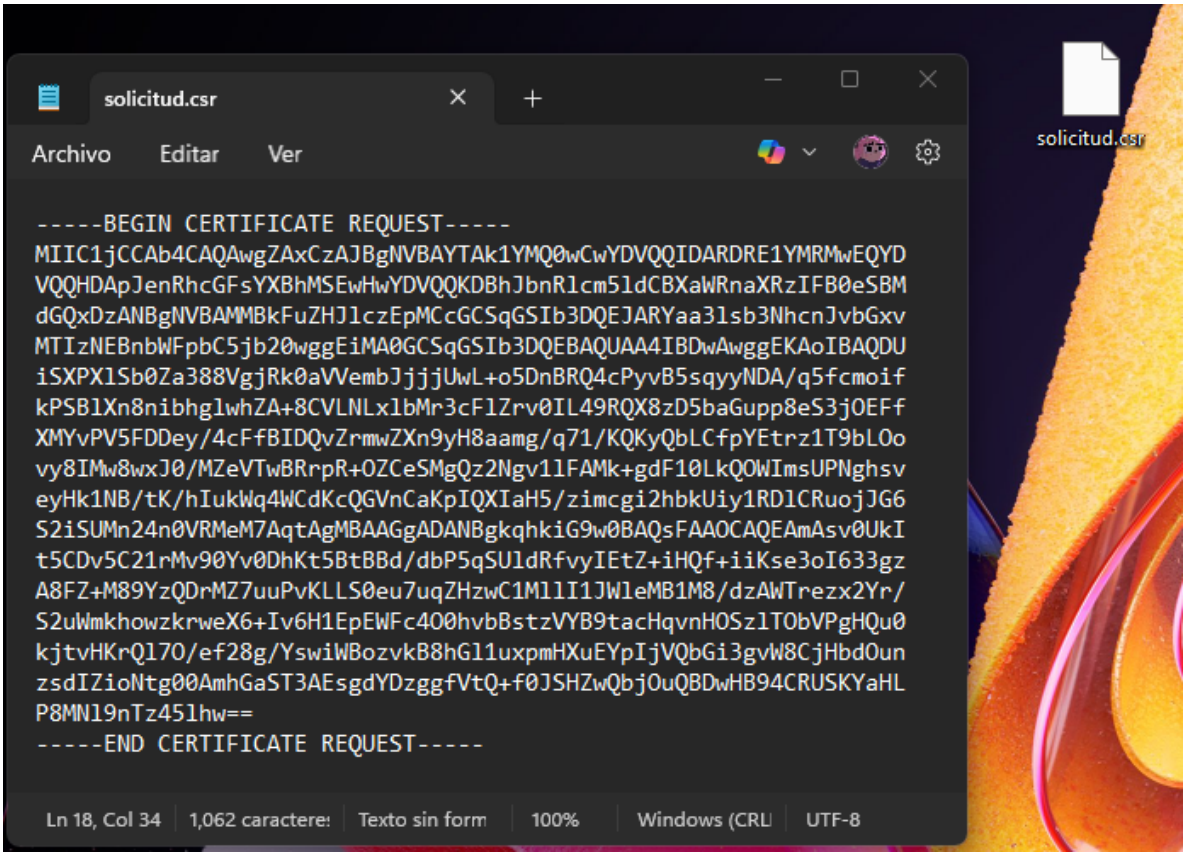


2. Obtener la firma

Para ello, lo haré con Getacert. Hay que dirigirse a la página de Getacert primero.



Después, abrí mi archivo recién generado “solicitud.csr” con un editor de texto, y copiar todo el contenido, y después, pegarlo en la página de Getacert, y después, descargué el archivo .cer:



getaCert

Decoded CSR - Summary

Attribute	Value
C	MX
ST	CDMX
L	Iztapalapa
O	Internet Widgits Pty Ltd
CN	Andres
emailAddress	kylosarrollo1234@gmail.com

your signed public certificate(.cer) : Andres-2025-11-22-215123.cer

Open your signed public certificate(.cer)

in pem format(.pem) : Andres-2025-11-22-215123.pem

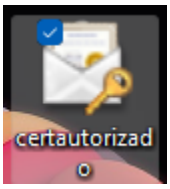
Open in pem format(.pem)

getaCert's public certificate/key(.cer) : getacert.cer

Open getaCert's public certificate/key(.cer)

This certificate is signed for 60 days. If you would like a 10 year certificate please Donate and we will send you a 10 year CSR signing page

Donate



3. Empaquetarlo con pkg12

Al igual que como el punto 3 de la práctica, empaqueté mi certificado firmado por una autoridad, con mi llave privada, esto mediante el comando “openssl pkcs12 -export -out certautorizado.p12 -inkey llaveprivada.pem -in certificadoautorizado.cer”

```
andik@KylosPC MINGW64 ~/Desktop
$ openssl pkcs12 -export -out certautorizado.p12 -inkey llaveprivada.pem -in
certificadoautorizado.cer
Enter Export Password:

Verifying - Enter Export Password:
```

Donde, al igual que en el punto 3, se me pide ponerle una contraseña, la cual, también es “kylos”.

Por último, el punto 5 de la práctica pide:

5. Verificar el certificado creado anteriormente.

Para ello, se utiliza el comando “openssl x509 -in certautorizado.p12 -text -noout”, el cual mostrará información relacionada al certificado digital x.509. En mi caso, se muestra de la siguiente manera:

```
andik@KylosPC MINGW64 ~/Desktop
$ openssl x509 -in certautorizado.p12 -text -noout
Enter pass phrase for PKCS12 import pass phrase:

Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 16142 (0x3f0e)
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=US, ST=Washington, L=Seattle, O=getaCert - www.getacert.com
        Validity
            Not Before: Nov 22 21:51:23 2025 GMT
            Not After : Jan 21 21:51:23 2026 GMT
        Subject: C=MX, ST=CDMX, L=Iztapalapa, O=Internet Widgits Pty Ltd, CN=
Andres, emailAddress=kylosarrollo1234@gmail.com
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:d4:89:25:cf:5e:54:9b:d1:96:b7:f3:c5:60:8d:
                    19:34:69:55:5e:99:b2:63:8e:35:30:2f:ea:39:0e:
                    70:51:43:87:0f:ca:f0:79:b2:ac:b2:34:30:3f:ab:
                    97:dc:9a:88:9f:90:f4:81:95:79:fc:9e:26:e1:82:
                    5c:21:64:0f:bc:09:52:cd:2f:19:5b:32:bd:dc:16:
                    56:6b:bf:42:0b:e3:d4:50:5f:cc:c3:e5:b6:86:ba:
                    9a:7c:79:2d:e3:38:41:5f:5c:c6:2f:3d:5e:45:0c:
                    37:b2:ff:87:05:7c:12:03:42:f6:6b:9b:06:57:9f:
                    dc:87:f1:a6:a6:83:fa:bb:d7:f2:90:2b:24:1b:2c:
                    27:e9:60:4b:6b:cf:54:fd:6c:b3:a8:bf:2f:08:33:
                    0f:30:c4:9d:3f:31:97:95:4f:00:51:ae:94:7e:39:
                    90:9e:48:c8:10:cf:63:60:bf:59:45:00:c9:3e:81:
                    d1:75:d0:b9:10:39:62:26:b1:43:cd:82:1b:2f:7b:
                    21:e4:d4:d0:7f:b4:af:e1:22:e9:16:ab:85:82:74:
                    a7:10:19:59:c2:68:aa:48:41:72:1a:1f:9f:f3:8a:
                    67:20:8b:68:5b:91:48:b2:d5:10:e5:09:1b:a8:8c:
                    91:ba:4b:68:92:50:c9:f6:e2:7d:15:44:c7:8c:ec:
                    0a:ad
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            Netscape Cert Type:
                SSL Client, SSL Server, S/MIME, Object Signing
            X509v3 Key Usage:
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Client Authentication, TLS Web Server Authentication
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        0a:18:6c:2e:bf:d8:65:df:80:fe:58:96:44:cc:e1:dc:61:40:
        8e:c8:e2:dc:26:db:ac:cf:c3:af:05:bf:d5:d7:a4:e8:a2:a4:
        dc:dc:80:c6:da:38:60:b6:32:e0:64:55:9a:f9:98:fb:e6:b6:
        a3:b0:36:50:9b:82:01:4c:af:74:16:c7:ea:4c:28:a0:f3:68:
        1a:b6:f8:cd:24:94:3e:84:9d:54:4d:ac:14:05:ed:11:c0:fa:
        34:34:94:18:f1:48:ff:59:d3:2e:a6:86:c8:a3:64:34:a2:cd:
```

Donde, en el párrafo “Issuer”, se confirma que efectivamente, getacert firmo el certificado que se encuentra en el archivo .pk12

Conclusión

A través de la realización de esta práctica, se logró configurar exitosamente un entorno de trabajo criptográfico en Windows mediante la implementación de OpenSSL a través de Git Bash. Se comprobó que, aunque el sistema operativo no incluye estas herramientas de forma nativa, el uso de gestores de paquetes como *Winget* facilita la integración de estándares industriales en equipos personales.

Respecto a la generación de claves asimétricas, se verificó el funcionamiento del algoritmo RSA para la creación de un par de llaves de 2048 bits. Fue posible constatar en la práctica la relación matemática y de dependencia que existe entre ambas: la clave pública no se crea de forma aislada, sino que se deriva directamente de la clave privada utilizando el subcomando rsa y las banderas de procesamiento adecuadas. Esto refuerza el concepto teórico de que la seguridad del sistema recae enteramente en el resguardo del archivo .pem privado.

Asimismo, la práctica permitió diferenciar los niveles de confianza en la infraestructura de clave pública (PKI). Se observó que, si bien técnicamente es posible generar certificados autofirmados (X.509) para entornos de prueba, estos carecen de validación externa. En contraste, el proceso de solicitud de firma (CSR) a una Autoridad Certificadora (CA) —como se realizó mediante los servicios web sugeridos— es indispensable para establecer una cadena de confianza legítima verificable por terceros.

Finalmente, la manipulación de formatos como PKCS#12 demostró ser esencial para la portabilidad de credenciales digitales, permitiendo encapsular tanto la identidad (el certificado) como la prueba de dicha identidad (la clave privada) en un único contenedor seguro, facilitando así su despliegue en servidores o aplicaciones finales.

Referencias

OpenSSL commands - OpenSSL Documentation. (s. f.).

https://docs.openssl.org/master/man1/#options